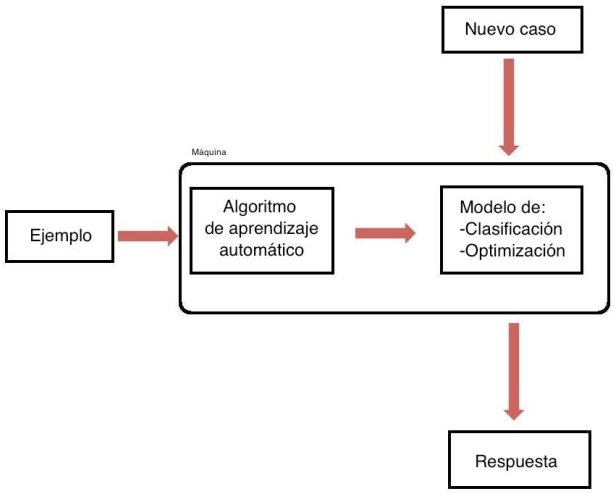
Primer Laboratorio

1. Introducción

Texto extraído de la Wikipedia:
https://es.wikipedia.org/wiki/Aprendizaje_autom%C3%A1tico
El aprendizaje automático (del inglés machine learning, ML), es el subcampo de las ciencias de la computación y una rama de la inteligencia artificial, cuyo objetivo es desarrollar técnicas que permitan que las computadoras aprendan. En el aprendizaje automático un computador observa datos, construye un modelo basado en esos datos y utiliza ese modelo a la vez como una hipótesis acerca del mundo y una pieza de software que puede resolver problemas.
El aprendizaje automático tiene una amplia gama de aplicaciones, incluyendo motores de búsqueda, diagnósticos médicos, detección de fraude en el uso de tarjetas de crédito, análisis de mercado para los diferentes sectores de actividad, clasificación de secuencias de ADN, reconocimiento del habla y del lenguaje escrito, juegos y robótica.



En esta práctica vamos a trabajar en un ejercicio de programación que simula un pequeño ejemplo de Aprendizaje Supervisado, que trata de encontrar un modelo para clasificar datos que contengan una etiqueta de interés. Ejemplos clásicos de este tipo de problemas son el triaje de pacientes, la concesión de un crédito bancario, la determinación de si un correo es spam, el precio de una vivienda, etc.
Todos estos problemas tienen un esqueleto común, un conjunto de registros con unos valores históricos (propiedades, atributos o *features* en inglés) y que tienen un parámetro distinguido (que llamaremos etiqueta) que es el que se quiere relacionar con los atributos.

Concretemos en un ejemplo, imaginemos que tenemos un histórico de pacientes de una enfermedad. A ese conjunto de personas se le sometió a una serie de pruebas diagnósticas (por ejemplo, análisis de sangre y de orina) que dieron lugar a unos valores de unos determinados atributos, añadiéndoles otros que podrían ser necesarios como género, edad, peso, etc. Cada paciente además fue etiquetado en su

enfermedad (por ejemplo “Leve” o “Grave”) por un tribunal médico que aplicó su conocimiento experto en la materia. Tenemos ese histórico guardado en un fichero csv en formato de tabla donde las filas son los pacientes y las columnas los valores de los atributos.

2. EJERCICIO

¿En qué consiste el problema? En encontrar un “algoritmo” tal que dado un nuevo paciente sea capaz de determinar si es probable que su situación se catalogue como Leve o Grave. Este tipo de problema se llama de ***Clasificación Supervisada*** ya que la etiqueta a encontrar pertenece a un conjunto discreto de valores (en nuestro caso Leve o Grave). Si la etiqueta es un valor real (no discreto), por ejemplo, el precio de una vivienda, entonces estamos hablando de un problema que se denomina de ***Regresión***.

Vamos a modelar ese problema desde un punto de vista de Clases en POO. Como simplificación vamos a suponer que todos los atributos son de tipo float, ya que el cálculo de distancias es distinto con atributos no numéricos.

<u>Estructura de Datos Básica de la clase ***Registro***:</u>
Objetivo: Comprender cómo representar un conjunto de puntos en un espacio n-dimensional.

- Un *constructor* que debe aceptar un vector (List) de números flotantes (valores de las propiedades o atributos) en su constructor.
- Debe tener una función *distancia_euclidea* para calcular la distancia Euclidea entre dos Registros.
- Debe tener una función *distancia_manhattan* para calcular la distancia Manhattan entre dos Registros.
- Debe tener una función *distancia_ponderada* para calcular una distancia ponderada. Para ello la función recibirá una lista de pesos y aplicará esta fórmula:

$$\text{Formula}=\sqrt{\sum w_i \cdot (p_i - q_i)^2}$$

- Implementar una función *calcula_distancia* que reciba una cadena de caracteres con alguna de estas palabras “euclídea”, “manhattan” o “ponderada” (por defecto será euclídea) y aplicará alguna de las funciones anteriores. Si se escoge ponderada debe tener los pesos si no, no.
- Implementar una función *normalizar* tal que dadas una lista de valores float con mínimos y una lista de valores float con máximos (ambos con la misma longitud que Registro) escala (normaliza) todos los valores de los atributos del Registro que invoca mediante la fórmula:

$$\left(x' = \frac{x - min}{max - min}\right)$$

  Donde max y min se refieren a los de cada dimensión. Esta función debe devolver otro Registro con los valores normalizados.

Por ejemplo, si el registro que invoca es [3.4,5.7] y la lista de mínimos es [2.7,3.3] y la de máximos [4.5,7.4] debe devolver [(3.4-2.7)/(4.5-2.7), (5.7-3.3)/(7.4-3.3)].

- Implementar una función *k_vecinos* que reciba una lista de Registro, un entero k, una cadena de caracteres con un tipo de distancia (por defecto será euclídea) y, cuando sea necesaria, una lista de pesos. Esta función debe devolver una lista con los índices de los k puntos de la lista de Registro que son los más cercanos al objeto que invoca con la distancia dada como parámetro.

Finalmente implemente un *test* con una función main para que se creen aleatoriamente unos 10 objetos de tipo Registro de dimensión 4 y pruebe la clase Registro.